2026/9/27

現行オンプレミス環境のAWS移行におけるターゲットアーキテクチャおよび技術設計提案書

「コンテナ基盤（ECS on Fargate）および完全IaC（Terraform）による次世代Webアプリケーションインフラの構築」

作成者：我有　顕彰

novel presentation

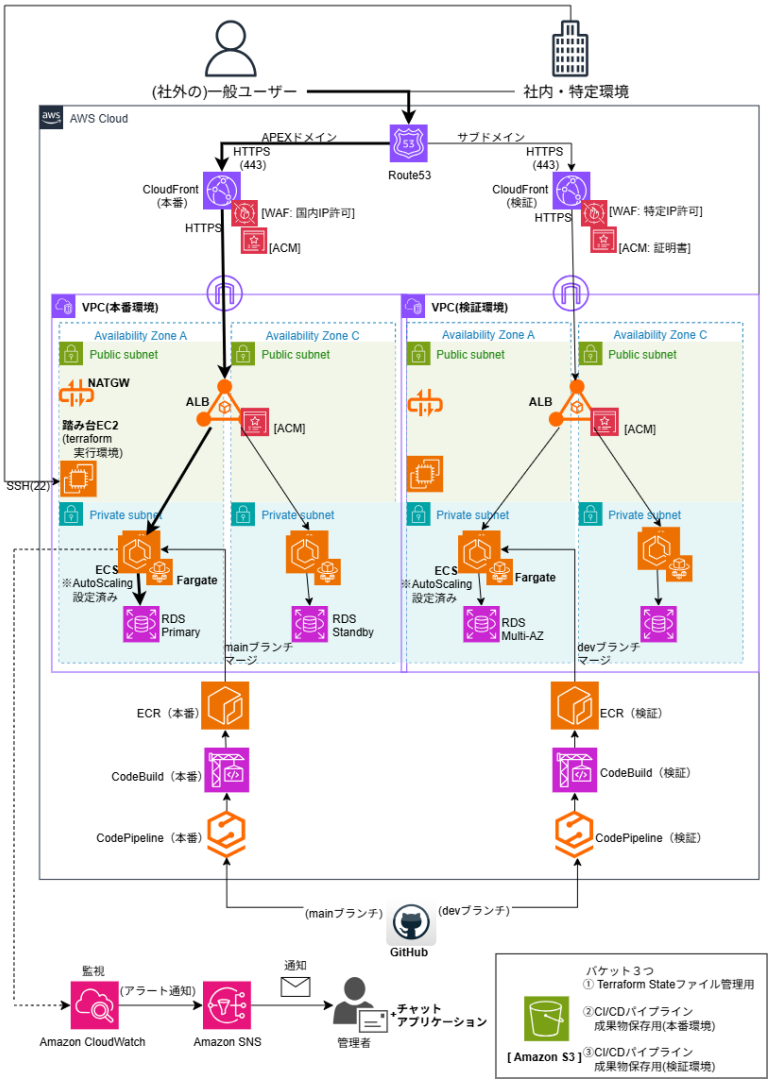
現行オンプレミス環境の技術的課題

- 物理ハードウェア（経年5年）の老朽化リスク
- リソース拡張性（スケーラビリティ）の限界
- 運用保守コストの逼迫

クラウド（AWS）移行における技術解決アプローチ

- **サーバーレス（Fargate）への移行：** 物理サーバーおよび仮想マシン（OSレイヤー）の管理をAWS側に移譲し、インフラの物理運用保守を完全にゼロ化。
- **多層防御による高可用性・耐障害性の確保：** 2つのアベイラビリティゾーン（マルチAZ）にインフラを分散配置し、アクセス集中時は自動増減（Auto Scaling）で処理能力を維持。
- **インフラのコード管理（IaC）と自動デプロイ：** インフラの構成変更をすべてコード化し、アプリケーションのリリースと連動して全自動でデプロイを実行する環境の構築。

AWS ターゲットアーキテクチャ定義（システム構成図）



- **エッジセキュリティ：** Route 53 から CloudFront + AWS WAF を経由し、HTTPS (443) 通信のみ許可するセキュリティ最優先の設計。

- **ネットワーク隔離**
  - アプリケーション（ECS）およびデータベース（RDS）は、パブリックIPを持たない「プライベートサブネット」に完全に隔離。
  - 本番環境と検証環境も別々のVPCに分けることでデプロイ先の間違いを防ぐ。

- **自動スケーリング：** 高負荷時は Application Auto Scaling（図内の左端タスク重なり部分）により、CPU使用率に応じてタスク数を自動で増減。

Terraform（IaC）による自動構築インフラのパラメータ設計

- **VPC / Subnet ネットワーク設計**： 2つのアベイラビリティゾーン（1a / 1c）にパブリック・プライベートサブネットを完全に分離・画定。

- **Amazon ECS / Fargate コンテナ設計**： パブリックIPを一切持たせない安全なプライベートサブネット内に、本番・検証用の各クラスター・サービスを独立して自動配置。

- **Amazon RDS データベース設計**： MySQLを用いた主系・副系のMulti-AZ構成を定義。2AZ冗長化により、データの保護と自動フェイルオーバーを確立。

※本番スペック算出根拠：最大4タスク（全体で2 vCPU）へのAuto Scaling負荷から逆算。
【ECS】Laravel最大消費メモリ（128MiB）の4並列処理分とシステム領域（300MiB）を合算した **300＋128×4＝812 MiB** から、Fargate規格の **0.5 vCPU / 1,024 MiB (1 GiB)** を算出・採用。
【RDS】最大16並列となるECSからの同時接続・大量クエリを安全に受け止めるため、検証用（micro）からメモリを2倍に拡張した **`db.t4g.small` (2 GiBメモリ)** を選定し、メモリ不足によるクラッシュを防止。

GitHub連携による完全自動デプロイメント（IaC）フロー

- **CodePipelineによるリリースの完全自動化：** GitHubの各ブランチ（mainマージ ➔ 本番 / devマージ ➔ 検証）へのプッシュをトリガーに、ビルドからECSデプロイまでをコンベア式に全自動実行。
- **CodeBuildによるコンテナイメージ化：** Dockerfile.deployment を用いてApache+PHP8.2環境のイメージを高速ビルドし、マイグレーション専用タスクを実行後、Amazon ECRへ安全にプッシュ。
- **本番・検証環境の分離：** 本番用（prod）と検証用（dev）のディレクトリをそれぞれ作成し、そこからTerraform（HCLコード）により各環境の全インフラを踏み台サーバーごと一括自動生成。